



projeto 2

Programação de Computadores - Experiência Prática 2



2 Identificação do aluno

SS

**Samuel dos Santos Escoval CRUZ_EAD_Cst em Análise e Desenvolvimento de
Sistemas_1A_20261**



3 Estruturação dos dados iniciais

Esboço da estrutura de dados

Estrutura de dados inicial (Estoque) # O dicionário permite acessar e atualizar os produtos rapidamente utilizando o nome como chave estoque = { "Notebook": [10, 3500.00],
"Mouse": [50, 80.00], "Teclado": [30, 150.00] }



Esboço do laço *while* (loop e Menu)

```
# Laço de repetição para manter o menu em execução while True: # Exibição do menu print("\n--- MENU DE ESTOQUE ---") print("1 - Visualizar Estoque Atual") print("2 - Registrar Entrada de Produto") print("3 - Registrar Saída de Produto") print("4 - Sair do Sistema") # Coleta da opção escolhida pelo usuário opcao = input("Escolha uma opção (1 a 4): ")
```



Implementação da visualização e entrada - (Código Opções 1 e 2)

```
# Opção 1 - Visualizar estoque if opcao == "1": print("\n--- PRODUTOS EM ESTOQUE ---") # Percorre o estoque exibindo os produtos for produto in estoque: dados =  
estoque[produto] quantidade = dados[0] preco = dados[1] print(f"Produto: {produto} | Quantidade: {quantidade} | Preço: R${preco:.2f}") # Opção 2 - Entrada de produto elif  
opcao == "2": print("\n--- ENTRADA DE PRODUTO ---") nome = input("Digite o nome do produto: ") # Conversão explícita de string para inteiro quantidade = int(input("Digite a  
quantidade que está entrando: ")) # Verifica se o produto existe no estoque if nome in estoque: estoque[nome][0] += quantidade print("Quantidade atualizada com sucesso!")  
else: print("Produto não encontrado.")
```



Implementação do processamento de saída (Código Opção 3)

Para desenvolver a lógica de saída de produtos, primeiro é necessário verificar se o produto informado pelo usuário existe no estoque. Essa validação evita erros ao tentar acessar um item inexistente. Caso o produto exista, o programa solicita a quantidade desejada e verifica se o estoque possui saldo suficiente para realizar a saída. Para isso, utilizei condicionais aninhadas, permitindo validar primeiro a existência do produto e depois a disponibilidade da quantidade. Se houver quantidade suficiente, o valor é reduzido do estoque. Caso contrário, o sistema informa que o estoque é insuficiente.

```
elif opcao == "3": print("\n--- SAÍDA DE PRODUTO ---") nome = input("Digite o nome do produto: ") # Verifica se o produto existe if nome not in estoque: print("Produto não encontrado") else: # Conversão explícita de string para inteiro quantidade = int(input("Digite a quantidade que está saindo: ")) qtd_atual = estoque[nome][0] # Verifica se existe quantidade suficiente if quantidade > qtd_atual: print("Estoque insuficiente") else: estoque[nome][0] = qtd_atual - quantidade print("Saída realizada com sucesso!")
```



Finalização da lógica condicional (Código Opção 4 e *fallback*)

A implementação de mecanismos de encerramento e validação é importante para garantir que o sistema funcione de maneira segura e organizada. O uso do `break` permite encerrar corretamente o laço `while True`, evitando que o programa continue executando indefinidamente após o usuário escolher sair do sistema. Além disso, o bloco `else` funciona como uma validação para comandos inválidos. Caso o usuário digite uma opção inexistente no menu, o sistema consegue informar o erro sem interromper a execução do programa, tornando a aplicação mais estável e amigável para o usuário.

```
elif opcao == "4": print("Encerrando o sistema. Até mais!") break else: print("Opção inválida! Tente novamente.")
```



Lista de boas práticas

- `qtd` → `quantidade_atual` O novo nome evita abreviações e deixa mais claro que a variável representa a quantidade atual do produto no estoque.
- `preco` → `preco_produto` O novo nome identifica melhor que o valor armazenado representa o preço do produto cadastrado no estoque.
- `qtd_atual` → `quantidade_estoque` O novo nome reduz ambiguidades e deixa mais claro que a variável representa a quantidade disponível no estoque.



Submissão do código-fonte

```
# Sistema simples de controle de estoque # Estrutura de dados inicial (Estoque) # O dicionário permite acessar e atualizar os produtos rapidamente utilizando o nome como
chave estoque = { "Notebook": [10, 3500.00], "Mouse": [50, 80.00], "Teclado": [30, 150.00] } # Laço de repetição principal do sistema while True: # Exibição do menu print("\n---
MENU DE ESTOQUE ---") print("1 - Visualizar Estoque Atual") print("2 - Registrar Entrada de Produto") print("3 - Registrar Saída de Produto") print("4 - Sair do Sistema") opcao =
input("Escolha uma opção (1 a 4): ") # Opção 1 - Visualização do estoque if opcao == "1": print("\n--- PRODUTOS EM ESTOQUE ---") # Percorre todos os produtos cadastrados
no estoque for nome_produto in estoque: dados_produto = estoque[nome_produto] quantidade_atual = dados_produto[0] preco_produto = dados_produto[1] print( f"Produto:
{nome_produto} | " f"Quantidade: {quantidade_atual} | " f"Preço: R${preco_produto:.2f}" ) # Opção 2 - Entrada de produtos elif opcao == "2": print("\n--- ENTRADA DE PRODUTO --
-") nome_produto = input("Digite o nome do produto: ") # Converte o valor digitado de string para inteiro quantidade_entrada = int( input("Digite a quantidade que está entrando:
") ) # Verifica se o produto existe no estoque if nome_produto in estoque: estoque[nome_produto][0] += quantidade_entrada print("Quantidade atualizada com sucesso!") else:
print("Produto não encontrado.") # Opção 3 - Saída de produtos elif opcao == "3": print("\n--- SAÍDA DE PRODUTO ---") nome_produto = input("Digite o nome do produto: ") #
Verifica se o produto informado existe no estoque if nome_produto not in estoque: print("Produto não encontrado.") else: # Converte o valor digitado de string para inteiro
quantidade_saida = int( input("Digite a quantidade que está saindo: ") ) quantidade_atual = estoque[nome_produto][0] # Verifica se existe quantidade suficiente em estoque if
quantidade_saida > quantidade_atual: print("Estoque insuficiente.") else: # Atualiza o estoque após a saída do produto estoque[nome_produto][0] -= quantidade_saida
print("Saída realizada com sucesso!") # Opção 4 - Encerramento do sistema elif opcao == "4": print("Encerrando o sistema. Até mais!") break # Tratamento para opções
inválidas else: print("Opção inválida! Tente novamente.")
```



Verificação das atividades realizadas

- Definição e carregamento da estrutura de dados inicial com ao menos 3 produtos.
- Implementação sustentada do laço de repetição `*while True*`.
- Apresentação limpa do menu e suas opções.
- Leitura de dados do estoque e iteração visando a formatação do relatório (Opção 1).
- Lógica de registro de entradas e verificação de chave/produto existente (Opção 2).
- Processamento de saída validando se o produto existe e se o saldo atende a demanda solicitada (Opção 3).
- Encerramento adequado do programa utilizando `*break*` (Opção 4).
- Mecanismo de contingência (`*else*`) alertando sobre opções de menu inexistentes.
- Incorporação de comentários analíticos e formatação PEP 8 no código entregue.



Reflexão

Considerando as competências avaliadas e as atividades realizadas, acredito que ainda possuo dificuldades e não compreendi completamente toda a linguagem Python. Para facilitar meu aprendizado, procurei dividir o desenvolvimento do sistema em etapas menores, organizando o projeto em partes como: armazenar produtos, criar o menu, manter o sistema em repetição contínua, permitir entrada e saída de produtos, atualizar o estoque e encerrar o programa corretamente. Essa divisão me ajudou a compreender melhor a lógica do sistema e a desenvolver cada funcionalidade separadamente antes de integrá-las ao programa principal. Durante as partes em que tive mais dúvidas, utilizei o auxílio de inteligência artificial para compreender melhor alguns conceitos, principalmente nas estruturas condicionais ``elif`` e ``else``, que inicialmente causaram certa confusão na aplicação prática. Acredito que, com mais exercícios e prática, conseguirei entender melhor como utilizar essas estruturas de forma mais natural e independente. Como ponto forte, considero que consegui compreender bem conceitos importantes como listas, variáveis, estruturas de armazenamento de dados e a utilização da função ``print()``, que já estão mais claros para mim durante o desenvolvimento dos programas. Além disso, percebi a importância da organização lógica do código, da utilização de comentários e da aplicação de boas práticas de programação, como nomes de variáveis mais descritivos e melhor estruturação do sistema. No geral, acredito que esta experiência prática contribuiu significativamente para meu desenvolvimento, principalmente no raciocínio lógico, na interpretação de problemas e na construção gradual de soluções em programação.

